Fulton Hogan Assignment

Software Modelling FINAL Report ENSE706

Fateh Bhular 23217987

Summary from Progress Report

The first phase of this assignment focused on the initial planning of the new Project Costing & Management system of Fulton Hogan. We focused on the three most important sub-departments which were **Project Management**, **Finance**, **and Operations**. Each of these departments have three important stakeholders each:

Project Management	**Finance**	**Operations**
Project Manager Project Coordinator Site Lead	Financial Controller Auditor Project Accountant	Site Foreman Heavy-Machine Operator General Labourer

During requirements elicitation, two stakeholders from each sub-department were interviewed about their current system "JD Edwards". It was noted that each user faced issues that reduced productivity or halted work all together. After all the interviews were finished, it was determined that their existing system suffered from data latency, a lack of real-time synchronisation, and data inconsistencies.

With the interviews, and some observations of JD Edwards online, the results were translated into functional and non-function requirements. These requirements were analysed thoroughly and focused on the main issues of the existing system being data integrity, system consistency, and communication between/in departments.

During UML Requirements Modelling, the system's behaviour was implemented using a **Use-Case Diagram**. This diagram mapped the relationships between actors-actions, and actions-actions. The final iteration of the user-case diagram was used as a blueprint for the **Class Diagram** later on, which focused on the structural frame of the system program. This diagram converted the different use-cases into concrete and abstract classes with labled interactions. The last iteration of the class diagram was a great improvement over the first iteration, as more classes were added to support previous inconsistencies. Design principles such as SRP, ISP, etc were improved throughout the iterations, and abstract classes were also improved.

During the iterations of the use-case and class diagrams, the **C# prototype** was being produced. The initial iteration was focused on the main concrete classes that were stated in the use-case descriptions section. During production, design risks were identified, such as high coupling, low cohesion, not following SRP, and more. The Agile cycle was implemented because it was the right fit, allowing improvements to be made during development and not at the end when everything is almost finished.

1. Refined Static and Dynamic Models

1.1 Evolution of the Class Diagram

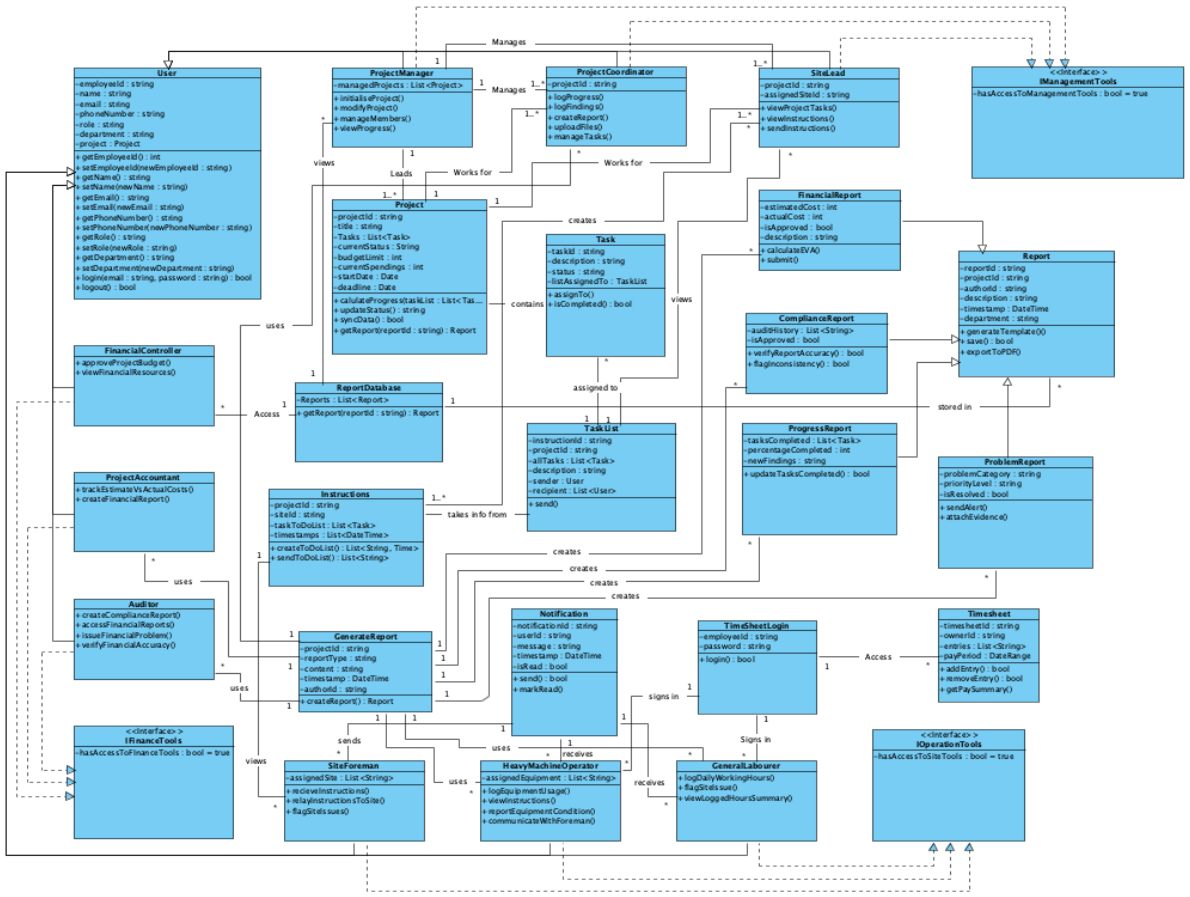

Figure 1: UML Initial Class Diagram

The initial Class diagram contained issues that were not fully resolved. Entities like the different employees, and different report classes were all mapped to concrete data classes which should've been abstract classes. This iteration was good enough that some people could understand what the system would do. However it still showed high coupling and low cohesion, and violated some SOLID design principles.

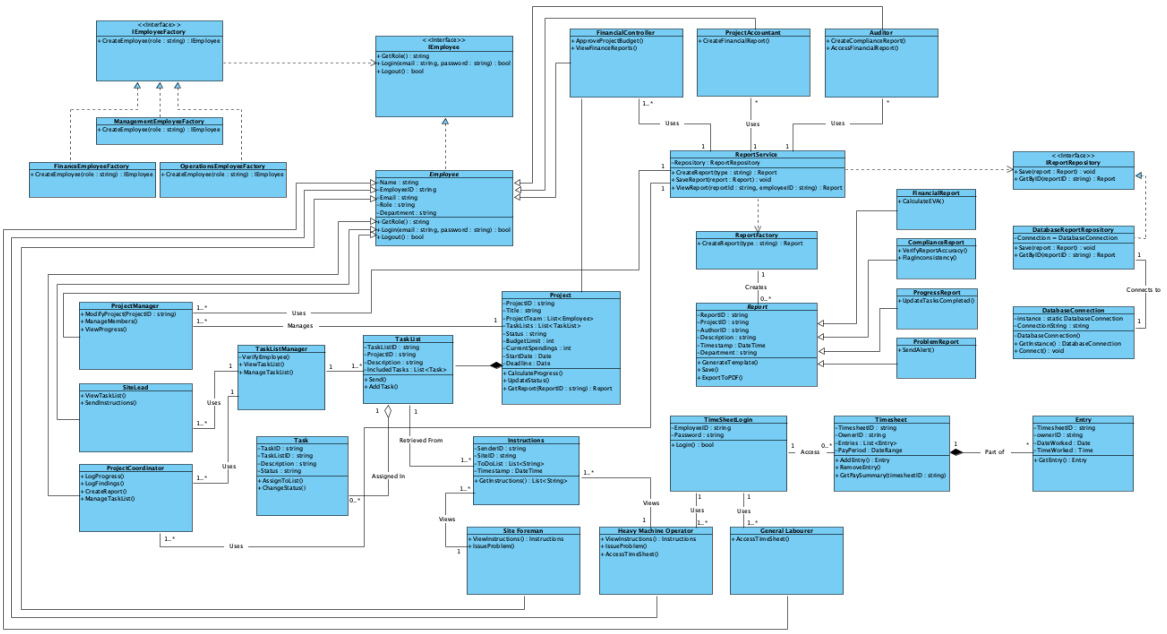

Figure 2: Refined UML Class Diagram

This Class diagram had many changes applied in order to make it into a low coupled system, where each section doesn't rely on another heavily. By implementing abstract classes and interfaces in the correct situations, this diagram didn't contain issues that the previous iteration had. Different departments and roles are now more independent in order to follow the rule of low coupling and high cohesion.

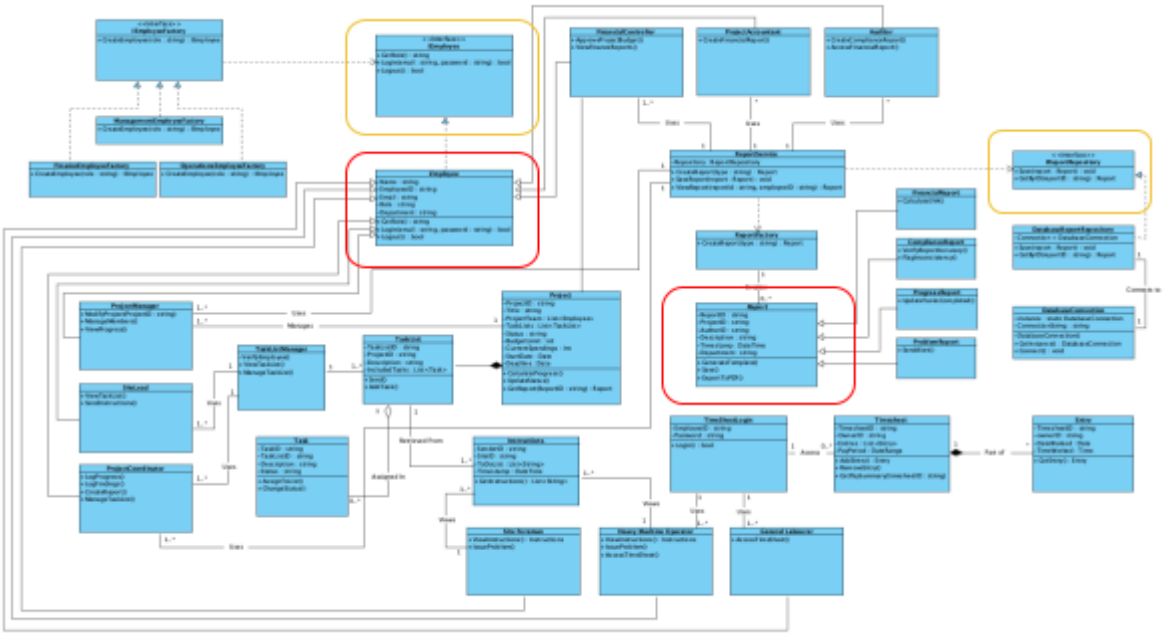

Figure 3: Class Diagram with highlighted abstract + interfaces

Some important changes are the *User* concrete classes becoming *Employee* abstract classes. In the previous iteration, the different employee concrete classes were mapped to a concrete data class. This was changed so that they map to an abstract class instead. The reasoning for this is that the *Employee* class is meant to be a representation of the different types of employees. It should not be an actual implementation which gets initialised in the program. Also abstract classes do not have fully implemented methods, which allows for different employees to have their own methods which can override the inherited method. Therefore making it abstract is the right call.

Looking at *Figure 1*, the interfaces that the different employee classes implemented, were not defined and in some cases were incorrect. They relied on boolean attributes, which have now been removed in the final iteration. Now there is an IEmployee interface which every employee sub-class implements. Interfaces should define the behavioural aspects of an object (what it does), not the structural aspect (what it has). Because the boolean was an attribute, it was incorrectly implemented into the model. Looking at *Figure 3*, the new employee interface only contains behavioural blueprints for the employee objects, which follows the correct interface structure.

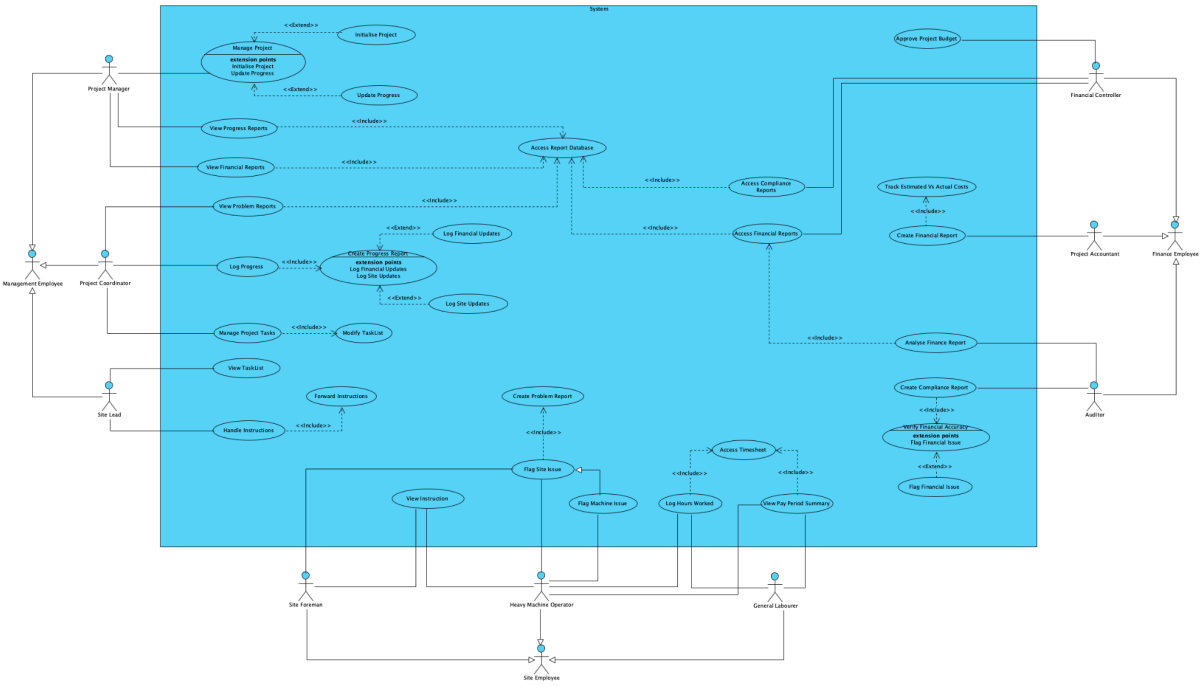

Figure 4: Use-Case Diagram for Project Costing & Management System

1.2 Sequence Diagrams

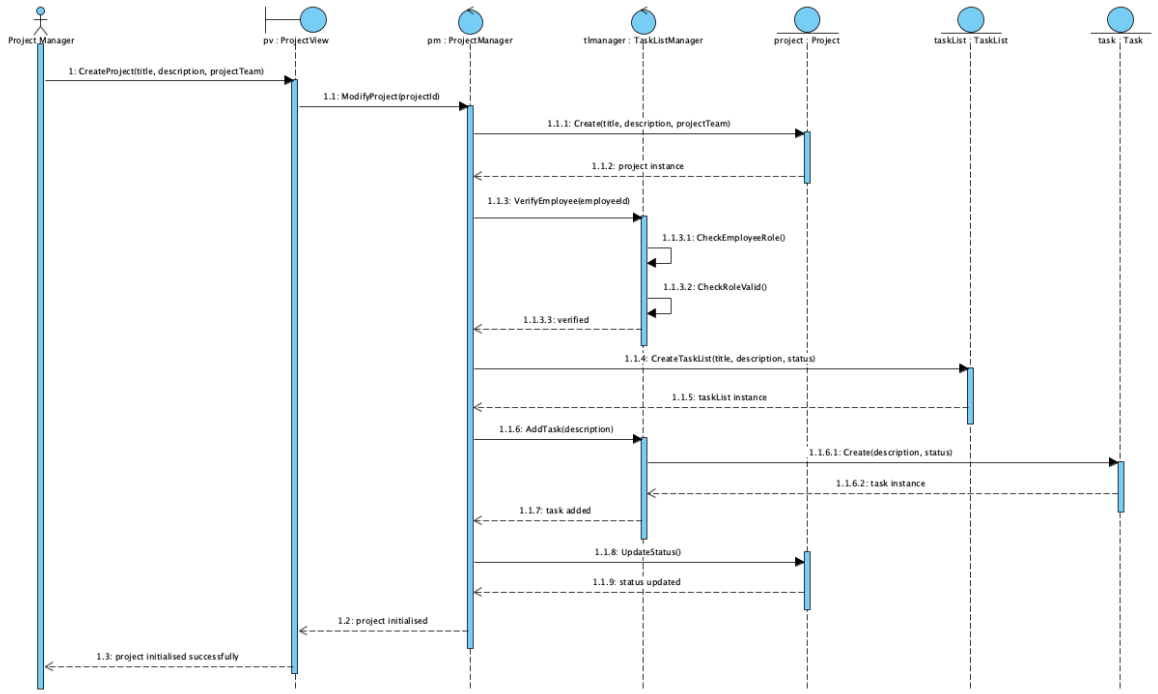

Figure 5: Sequence Diagram for "Modifying a project"

The sequence diagram displayed above (*Figure 4*) models the *Initialise Project* extention point from the Manage Project use-case. A boundary has been applied which makes it so the Project Manager communicates with the ProjectView UI, which further talks to the Manager control objects. A self-message has been added to TaskListManager to model the validation. This is performed to check the user's role before any changes are made to the project. Entity objects such as TaskList and Task are destroyed at the end of the scenario.

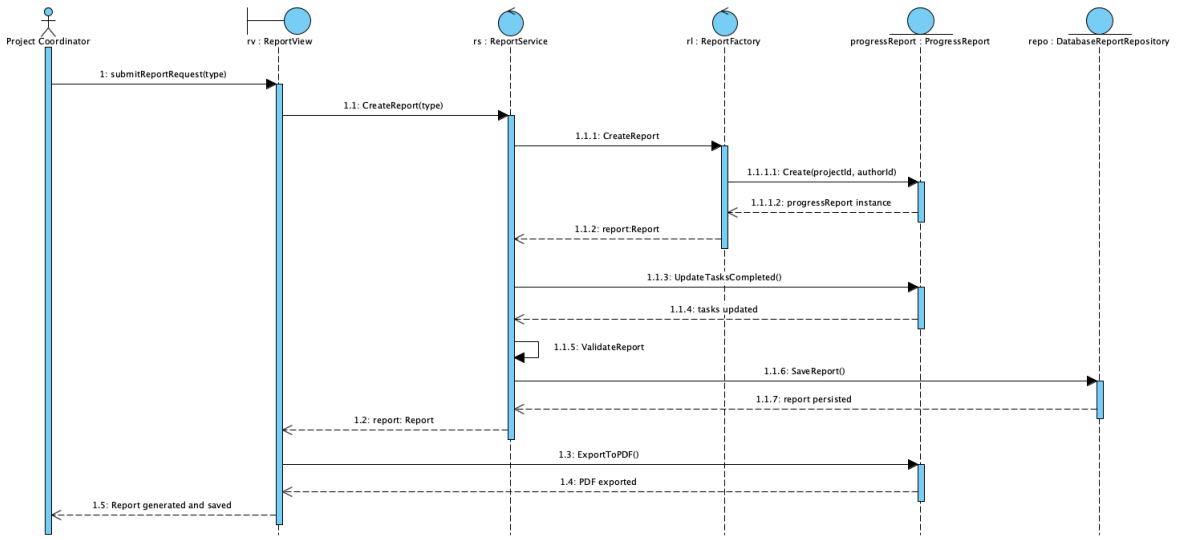

Figure 6: Sequence Diagram for "Generating a Progress Report"

The sequence diagram above (*Figure 5*) models the report generation flow from the Log Progress use-case. The Factory-design pattern was applied where ReportService delegates object creation to ReportFactory, this was modelled to show how the service is decoupled from the concrete report types. There is a self-message on ReportService which represents the Service checking to see if the report is valid to be persisted into the database. DatabaseReportRepository handles storing reports. In *Figure 3*, you can see this is handled through the IReportRepository interface.ReportFactory and ProgressReport objects are destroyed when they have done their job.

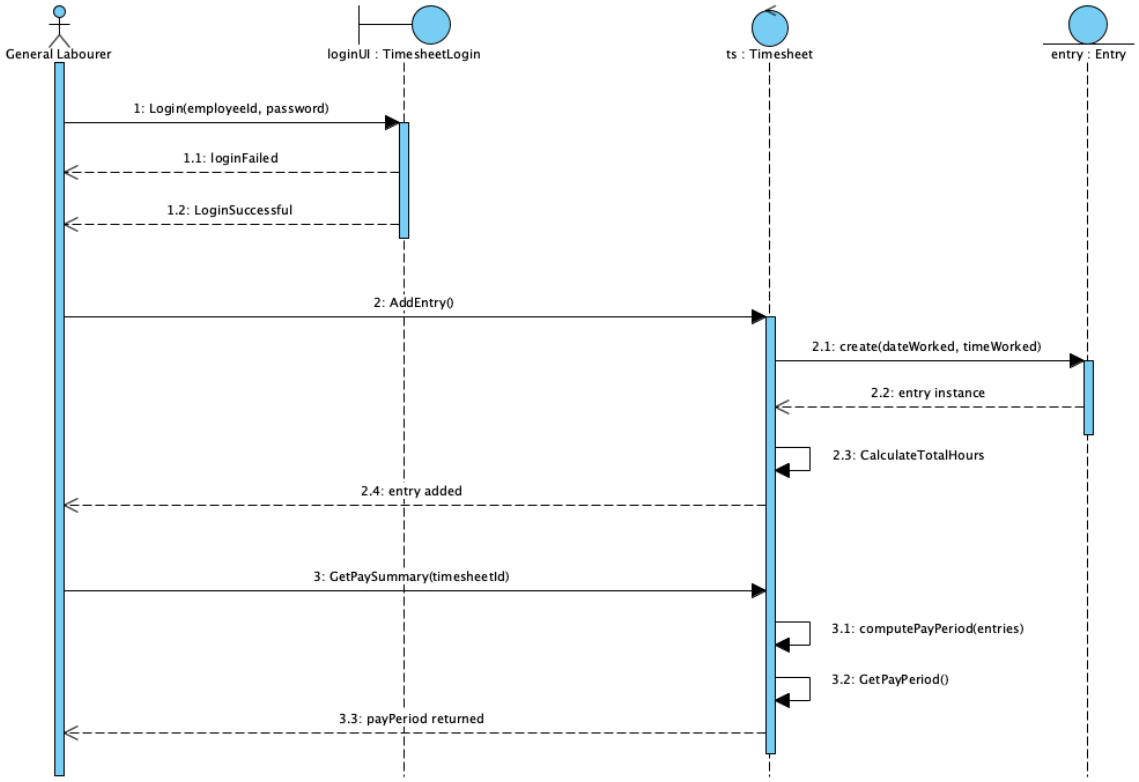

Figure 7: Sequence Diagram for "Creating a timesheet entry"

This sequence diagram (*Figure 6*) models the flow for accessing the timesheet and adding an entry. An Alt fragment was used to handle login success and failure. Self-messages are used in the Timesheet control object to (1) compute total hours (2) compute the pay period and look at it. This is represented as a self-message in order to keep calculations encapsulated. The Entry entity is destroyed after it has been created and added to the timesheet.

1.3 Activity Diagrams

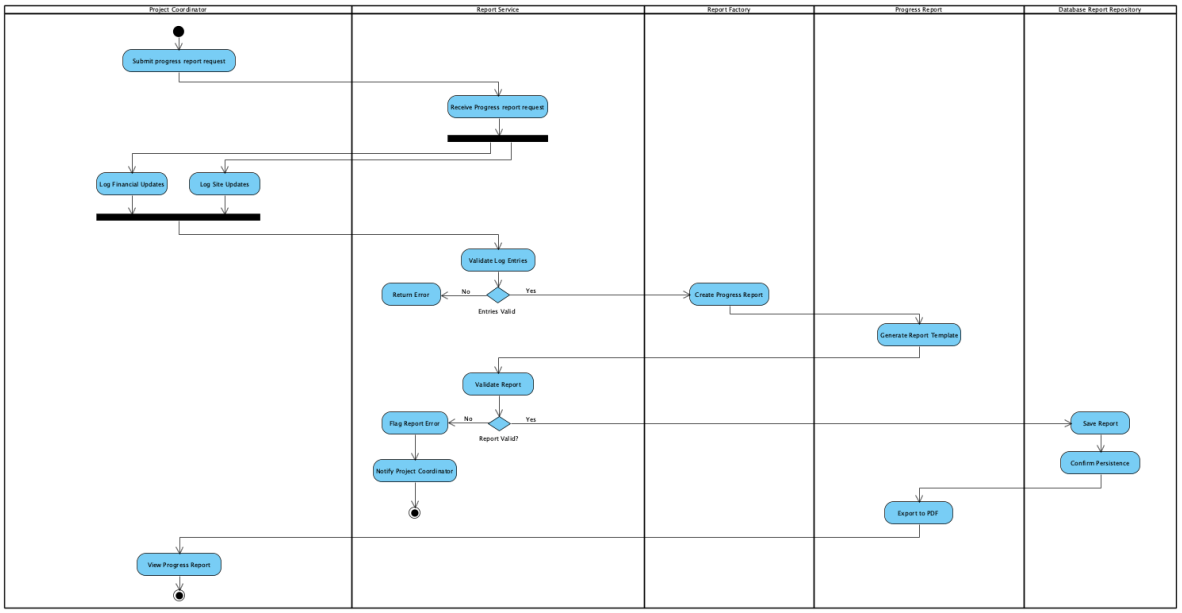

Figure 8: Activity Diagram for "Logging Progress and Generating Report"

The activity diagram in *Figure 7* models the flow of progress report generation. There are 5 swimlanes each representing a class or object that has a responsibility in this flow of events. A fork node and join node were used to show the options a user has. They can choose to log financial or site updates into the report. This is joined later and validated by the ReportService class. For conditions, the decision node was used to show the options that could occur as a reaction to an event (like an if-else statement). For example, validation of the report had 2 different outcomes (1) If not validated, then the user would be flagged (2) If validated, the report would be saved into the database. If the report was persisted into the database, a PDF would also be generated for easy viewing for any viewers. Both fork, and decision nodes are important to a sequence diagram, because it allows for diagrams that are easier to understand. Outsiders wouldn't know what an if-else statement is, so these nodes display it in a way that is easy to visualise.

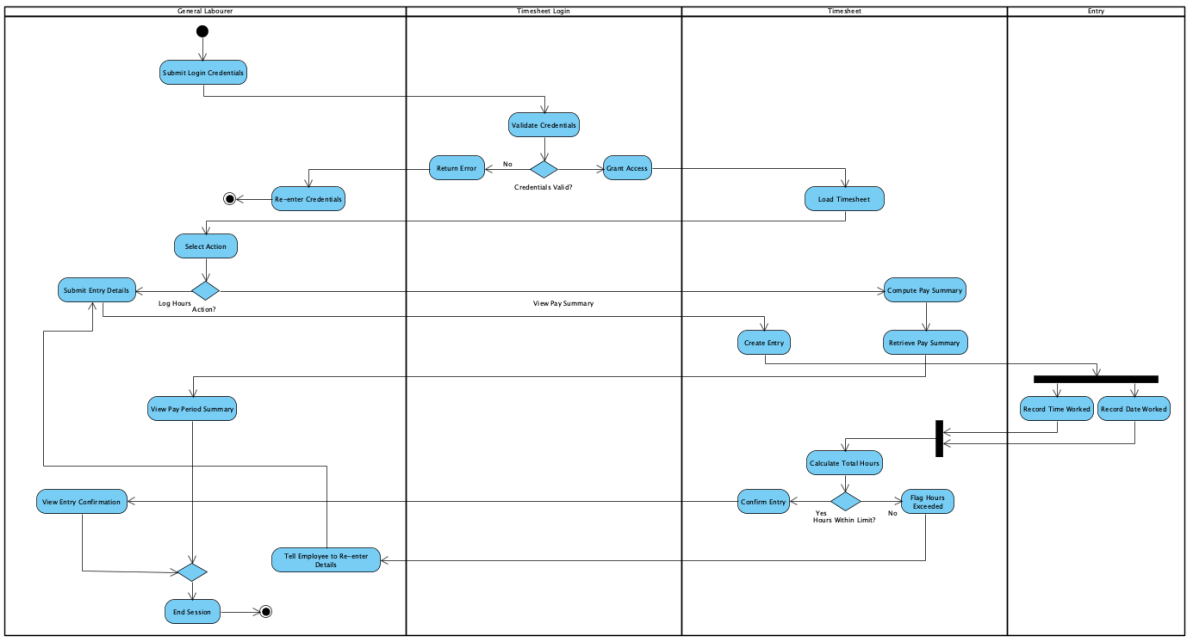

Figure 9: Activity Diagram for "Timesheet Operation Flow"

The diagram in *Figure 8* models the Timesheet Operation Flow, which covers both the Log Hours Worked, and View Pay Period Summary use-cases (*Figure 9*). There are four swimlanes representing General Labourer, TimesheetLogin, Timesheet, and Entry. All four of these objects/classes play a role in this operation flow. A decision node was used for validating credentials because there are two possible cases (1) employee enters correct details, so they can access timesheet (2) employee enteres incorrect details, they are prompted to sign in again. Because there are two different outcomes from the Validate Credentails action, a decision node is more suitable than a fork node. However, a fork node was used to create an Entry. Because the system has to record two things that are separate, it could occur at the same time to save time. When both actions are completed, they go into a join node and the system can then perform calculations.